

Hash Functions

Input m : binary string

Output $h(m)$:

- "Short" (n-bit) binary string (aka **message digest** or hash)

Efficiently computable

Applications: cryptography, security, efficiency

Keyed $h_k(m)$, where the key is public, or unkeyed $h(m)$

Motivation: Hashing for Efficiency

Input: large set (e.g. ints or strs)

Goal: map "randomly" to few bins

Security Goal: Collision Resistance:

A **collision**: two inputs (e.g. names) with the same hash: $h('Bob') = h('Phil')$

Every hash has collisions, since $|\text{input}| \gg |\text{output}|$

Collision resistance: hard to **find** these collisions

- Note: attacker can always try names randomly until a collision is found
- But this should be ineffective: must try about (on average) N names (number of bins)

Collision Resistant Hash Function (CRHF)

h is CRHF if it is hard to find collisions $h(x) = h(x')$ for $x \neq x'$

- Note: attacker can always try names randomly until collision is found
- But this should be ineffective: must try about $|Range|$ values

Definition (Keyless Collision Resistant Hash Function (CRHF)). A keyless hash function $h^n(\cdot) : \{0, 1\}^* \rightarrow \{0, 1\}^n$ is *collision-resistant* if for every efficient (PPT) algorithm A , the advantage $\varepsilon_{h,A}^{CRHF}(n)$ is negligible in n , i.e. smaller than any positive polynomial for sufficiently large n (as $n \rightarrow \infty$), where:

$$\varepsilon_{h,A}^{CRHF}(n) = \Pr[(x, x') \leftarrow A(1^n) \text{ s.t. } (x \neq x') \wedge (h^n(x) = h^n(x'))]$$

Keyless CRHF Do Not Exist

$|Range| \ll |Domain|$ so there is a collision where $h(x') = h(x), x \neq x'$

Keys are powerful because they allow you to change the terms on your adversary.

For a keyless CRHF there is a PPT algorithm A that can always output a collision: $A(1^n) = \{return\ x, x'\}$

- Proof: intuitively, since the function is fixed (same input-output mapping), a collision instance can be hardcoded into the attacker algorithm and just output that collision and win the security game.

Keyed CRHF

$k \xleftarrow{\$} \{0, 1\}^n \leftarrow \text{Adversary} \leftarrow (x, x') \text{ If } x \neq x' \text{ and } h(x) = h(x')$

Generic Collision Attacks

An attacker that runs in exponential time can always find a collision (i.e. non-PPT attacker)

- Easy: find collisions in 2^n time by trying $2^n + 1$ distinct inputs (compute their hash and locate a collision)

An attacker finds a collision with 2^{-n} probability (negligible probability).

- Choose x and x' at random and check if they produce a collision.

The Birthday Paradox

The birthday paradox states that the expected number q of hashes (i.e. hash queries) until a collision is found in $O(2^{\frac{n}{2}})$ not $O(2^n)$. It is $q \lesssim 2^{n/2} * \sqrt{\frac{\pi}{2}} \lesssim 1.254 * 2^{n/2}$

For 80 bits of effective security, use $n = 160$

- So to defend against an attacker who can perform 2^{80} hashes set the digest length to at least 160 bits (so the range has a size of 2^{160} digests)

The Birthday Attack

Probability of NO birthday-collision:

- Two people: $(364/365)$
- Three people $(364/365)(364/365)$
- ...
- n people: $\prod_{i=1}^{n-1} \frac{365-i}{365}$
- So for 23 individuals, the probability of a birthday collision is about .5

Collision-Resistance: Applications

Integrity (of object/file/message)

- Send $hash(m)$ securely to validate m
- Later, we will see how a hash function can be used to construct a MAC (called HMAC)

Hash-then-Sign

- Instead of signing m , sign $hash(m)$ (more efficient!)

Blockchains